

DDalphaAMG 软件库全面分析报告

opencode analy

2026-08-17

摘要

本报告对格点 QCD 多重网格求解器库 **DDalphaAMG** (位于 [/root/PyQCU/refer/git-rep/DDalphaAMG](#)) 进行只读全面分析。该库以聚合型代数多重网格 (AMG) 方法求解 Wilson-Clover 狄拉克方程 $D_W(U, m_0)\psi = \eta$, 平滑器采用 Schwarz 交替过程 (SAP), 并含 CUDA 移植、SSE/AVX 向量化与 MPI/OpenMP 并行。报告覆盖三视角: **主体结构** (入口/核心/GPU/测试分层)、**各部分关系** (sed 模板实例化链、‘src $\rightarrow$ CUDA’ 分派链、‘ini $\rightarrow$ run $\rightarrow$ dd_alpha_amg’ 运行链)、**项目思路** (模板化精度生成、自适应 setup、K-cycle 设计动机); 并对”从编译到运行”完整 workflow 按 A 框架 15 步重现。本快照非独立 git 仓库 (共享父仓库 [/root/PyQCU/.git](#)), 且缺少 CPU 侧 .c 实现文件与 Makefile, 相关证据以现有 *_generic.h 声明与 src/gpu/*.cu 实现为准。

目录

1 仓库概览

1.1 目录结构

```
1 DDalphaAMG/
2 |-- README / COPYING / CREDITS / NEWS      # 说明与许可
3 |-- MakeSettings.mk(.template)             # 用户编译配置
4 |-- double.sed / float.sed                 # 精度实例化脚本
5 |-- compile_modules.sh / setup_env_and_compile.sh
6 |-- run / submit_E250_*.job / sub8to4.sh    # 运行/作业脚本
7 |-- test.sh / test_wrapper.sh              # 一键测试
8 |-- doxygen.conf                           # Doxygen 配置
9 |-- src/                                    # 核心源码 (79 头 + gpu/ 18 .cu + proxies/)
10 |-- test/                                  # 集成测试与 gtest (26 个 ini + CUDA 组件测试)
11 |-- conf/                                  # 预置规范场配置 (4^4 / 8^4)
12 |-- doc/                                   # 用户手册 user_doc.tex + bib
13 |-- docs/                                  # 本报告与历史 pure 报告
14 |-- ini/ repro/                            # 样例参数与复现实验
15 |-- dependencies/ # BLAS/LAPACK/BLACS/ScaLAPACK/QMP/QIO 安装脚本
16 |-- include/ lib/ bin/ output/             # 构建产物占位 (空)
```

指标	实测值
跟踪文件数	201 (含 test/conf/docs)
src/ 全部	129 文件 / 19060 行
src/gpu/ (CUDA)	43 文件 / 10424 行 (含 18 个.cu)
src/*.h (顶层头)	79 文件 / 8503 行
test/	29 文件 / 3709 行
最大单文件	src/gpu/cuda_oddeven_generic.cu 4532 行
独立 git 仓库	否 (共享 /root/PyQCU/.git, 快照形态)

表 1: 仓库实测统计 (数据来源: find/wc/git 实测)

1.2 规模统计与 git 信息

2 主体结构

层次	目录	职责
入口层	src/top_level.h 等	顶层驱动: wilson_driver/solve/solve_driver (外求解器路径选择)
核心层	src/	AMG 算法: Dirac 算子、偶奇预处理、自适应 setup、聚合/插值、粗算子、SAP、V/K-cycle、Krylov 求解器
GPU 层	src/gpu/	CUDA 内核: Wilson+Clover、块级偶奇 Schur、粗算子、ghost 通信、FGMRES/局部 MINRES
分派层	src/proxies/	CPU/GPU 按 CUDA_OPT 编译宏分派 (如 dirac_proxy_generic.h)
向量化层	src/sse_*.h	SSE/AVX 双缓冲向量化实现 (sse_dirac.h 548 行)
并行层	src/communication_generic.h 等	MPI halo 交换 (8 方向 boundary table + 双缓冲) 与 OpenMP 嵌套线程
配置层	ini/, sample*.ini 等	输入参数文件: 几何/精度/方法/多级参数
构建层	MakeSettings.mk, *.sed	编译配置、精度模板实例化、环境装配 (setup_env_and_compile.sh)
工具层	dependencies/, run	依赖安装、配置切分、进程数计算与启动
文档层	doc/user_doc.tex 等	用户手册、Doxygen 项目配置
测试层	test/	集成回归 (integration_test.sh) + 26 个 ini + gtest

层次	目录	职责
----	----	----

表 2: 主体结构分层 (数据来源: 全库索引实测)

要点

要点: 本库以”头文件模板 + sed 实例化”为核心组织方式——同一算法以 `*_generic.h` 书写, 经 `double.sed/float.sed` 替换 `PRECISION` 占位符生成 `*_double.h/*_float.h`; CPU 侧仅保留声明与 `static inline`, 完整实现集中在 `CUDA .cu` 内核。

3 各部分关系

3.1 精度实例化链 (核心生成机制)

要点

依赖主链: `*_generic.h` (含 `PRECISION` 占位符) $\xrightarrow{\text{double.sed/float.sed}}$ `*_double.h/*_float.h`
 $\xleftarrow{\text{src/main.h}}$ 头文件装配

证据: `double.sed:1` 为 `s/PRECISION/double/g`; `src/main.h:91-92` 引用 `main_pre_def_float.h/double.h`, `src/main.h:106-243` 完成全部头文件装配。双精度与单精度两套实例可共存 (`level_struct.h:13-99` 中 `op_double/op_float` 并存), 支撑混合精度求解 (`doc/user_doc.tex:84`)。

3.2 CPU/GPU 分派链

要点

依赖主链: `src/proxies/*.h` (按 `CUDA_OPT` 宏) $\rightarrow$ `d_plus_clover_cpu` (CPU 声明) 或 `cuda_d_plus_clover` (GPU 内核)

证据: `src/proxies/dirac_proxy_generic.h:1-8` 按 `CUDA_OPT` 选择 CPU/GPU 实现; `MakeSettings.mk:17-41` 的 `CUDA_ENABLER/SSE_ENABLER/AVX_ENABLER` 开关控制编译宏。**注意:** 本快照中 CPU `.c` 实现缺失, `*_generic.h` 仅保留声明与 `static inline` 工具函数, 完整逻辑需对照 `CUDA .cu`。

3.3 运行链

要点

依赖主链: `<input>.ini` (几何/精度/方法参数) $\rightarrow$ `run` (计算 MPI 进程数) $\rightarrow$ `mpirun dd_alpha_amg <ini>` $\rightarrow$ 输出到 `output/`

证据: `run:87-99` 由 $np = \prod_{\mu} \text{global/local}$ 计算进程数 (公式亦见 `doc/user_doc.tex:59-60`); `run:196-210` 执行 `mpirun`。

3.4 文件依赖关系汇总

源 (A)	目标 (B)	关系类型
<code>double.sed/float.sed</code>	<code>*_double.h/*_float.h</code>	生成 (s/PRECISION/.../g)
<code>src/main.h</code>	全部 <code>*_PRECISION.h</code>	包含/装配
<code>src/gpu/cuda/*.generic.*</code>	<code>src/gpu/*.cu</code>	模板实例化与 kernel 发射
<code>src/proxies/*.h</code>	CPU 声明 / CUDA 内核	编译期分派
<code>MakeSettings.mk</code>	构建	配置驱动
<code>sample*.ini</code>	<code>dd_alpha_amg</code>	运行时输入
<code>run</code>	<code>mpirun</code>	进程数计算 + 启动
<code>doc/user_doc.tex</code>	<code>user_doc.pdf</code>	文档生成 (make documentation)

表 3: 各部分关系汇总 (数据来源: 源码与配置实测)

4 项目思路

4.1 设计意图

DDalphaAMG 以“聚合型代数多重网格 + Schwarz 平滑器”为核心竞争力: 针对 Wilson-Clover 算子近零模导致 Krylov 求解器收敛退化的问题, 用自适应 `setup` 构造插值算子 (`src/setup_generic.h:30 iterative_PRECISION_setup`), 经 Galerkin 三重积 $P^{\dagger}DP$ 构造粗算子 (`src/coarse_operator_generic.h:31 coarse_operator_PRECISION_setup`), 每层用 SAP 平滑 (`src/schwarz_generic.h:49-56 additive/red-black/sixteen-color` 三变体), 实现格子无关的收敛率。设计上强调 **可移植性**: 同一算法头经精度 `sed` 与 CPU/GPU 分派层适配多架构 (MPI/OpenMP/SSE/CUDA)。

项目思路

项目思路核心总结: 一个“算法模板化、实现分派化、精度实例化”的格点 QCD 多重网格求解器——算法逻辑写一遍 (`*_generic.h`), 编译期决定精度 (`double/float`) 与后端 (CPU/SSE/CUDA), 运行时由 `ini` 决定多级几何与方法。

4.2 演进脉络

NEWS:1-9 记录 v1606 (首版) 与 v1701 (多文件配置 IO) 两个里程碑; doc/user_doc.tex:50 说明 v1701 新增多文件配置读取。当前版本的核心新增为 CUDA 移植 (README:4 注明 Wilson 等关键部分已移植 CUDA)。CREDITS:1-15 列出八位开发者。repro/matthaei2023/ 为 Matthaei 2023 论文复现参数, 反映”算法论文-复现实验”的持续演进路径。

4.3 典型 workflow

从零运行一次求解的标准流程:

1. 改 compile_modules.sh/MakeSettings.mk (模块与编译器路径);
2. . setup_env_and_compile.sh (source 后 make 编译);
3. 配置 sample_E250_CPU/GPU.ini 的几何/精度/方法参数;
4. sbatch submit_E250_*.job (SLURM 提交) 或 run <ini> (本地);
5. 结果写入 output/, 必要时开 -DPROFILING/-DTRACK_RES 等 CFLAG 调优。

证据: README:6-13 (推荐用法)、doc/user_doc.tex:35-40 (运行)、doc/user_doc.tex:94-107 (CFLAG)。

5 主题分析——工作全流程重现 (A 代码工作框架)

本节将”用 DDalphaAMG 完成一次 Wilson-Clover 求解”作为一项具体代码工作, 按 A 框架 15 步完整重现。由于本快照不可构建运行, 实测执行步骤 (A7/A8/A9) 标注未验证, 以文档与源码证据推断。

5.1 A1 目的与前期准备

目的: 为格点 QCD 的 Wilson-Clover 狄拉克方程 $D_W(U, m_0)\psi = \eta$ 提供快速反演器 (doc/user_doc.tex:21-26)。**前置条件:** C 编译器、MPI 库、BLAS/LAPACK (可经 dependencies/脚本安装 LAPACK 3.9.0、BLACS/ScaLAPACK、USQCD QMP/QIO)。证据: doc/user_doc.tex:22、dependencies/install_dependencies_01.sh。

5.2 A2 输入是什么

- 规范场配置 U (DDalphaAMG 自有格式/LIME/多文件, doc/user_doc.tex:47-50);
- 参数文件 .ini: 配置路径、各层 global/local/block lattice、test vectors 数、setup 迭代数、 m_0 、 c_{sw} 、method、mixed precision、kcycle 等 (doc/user_doc.tex:53-89);
- 右端项 η (由求解入口构建)。